THE UNIVERSITY OF ALABAMA IN HUNTSVILLE

# Advanced Logic Design Lab

**Laboratory Assignment #X**

**I$^2$C Communication**

(*X% of Final Grade*)

## 1 Theory

### 1.1 Inter-Integrated Circuit (I$^2$C) Communication Fundamentals

Inter-Integrated Circuit (I$^2$C) is a synchronous serial communication protocol designed for short-distance communication between integrated circuits on the same board. It is widely used to connect sensors, memory devices, real-time clocks, GPIO expanders, and other peripherals to a microcontroller or FPGA. I$^2$C uses a shared bus architecture in which multiple devices can communicate over the same two signal lines. This reduces pin count and routing complexity compared to parallel interfaces or point-to-point serial links, while still supporting a structured, address-based communication model.

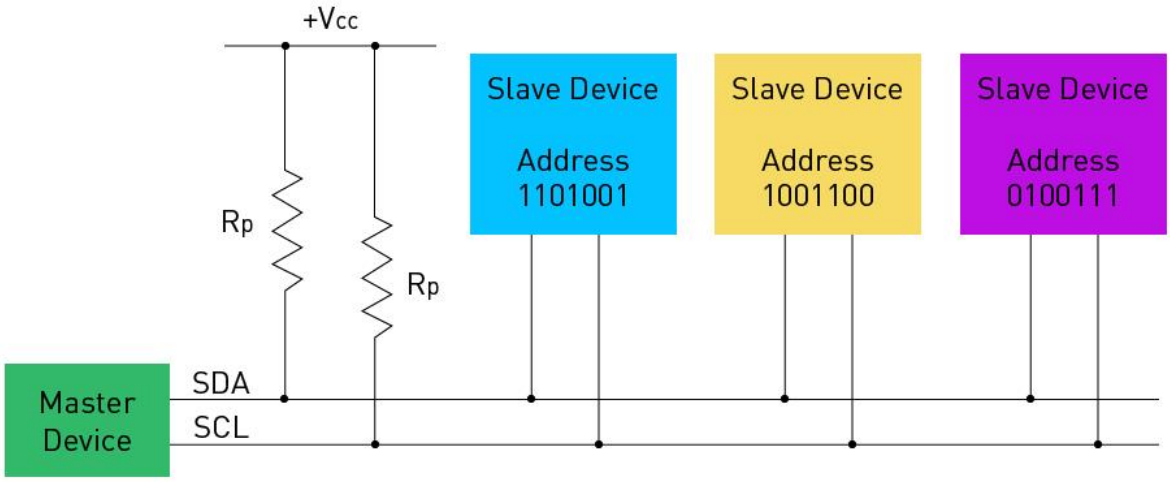



Figure 1: Conceptual I$^2$C bus showing shared `SCL` and `SDA` lines with one master and multiple slave devices.

### 1.1.1 Bus Signals and Open-Drain Operation

An I$^2$C bus uses two lines: `SCL` (serial clock) and `SDA` (serial data). The clock line is driven by the bus master and defines when data is valid. The data line carries addresses, control bits, and payload bytes in both directions depending on whether the master is writing to or reading from a slave.

A key feature of I$^2$C is its open-drain (open-collector) electrical interface. Devices do not actively drive logic-high levels. Instead, external pull-up resistors hold `SCL` and `SDA` high when no device is pulling them low. Any device can assert a logic low by sinking current to ground. This wired-AND behavior allows multiple devices to share the bus without electrical contention, since no device forces a high level against another device driving low. In practice, the pull-up resistors and bus capacitance determine rise times, which places limits on maximum bus speed and overall signal integrity.

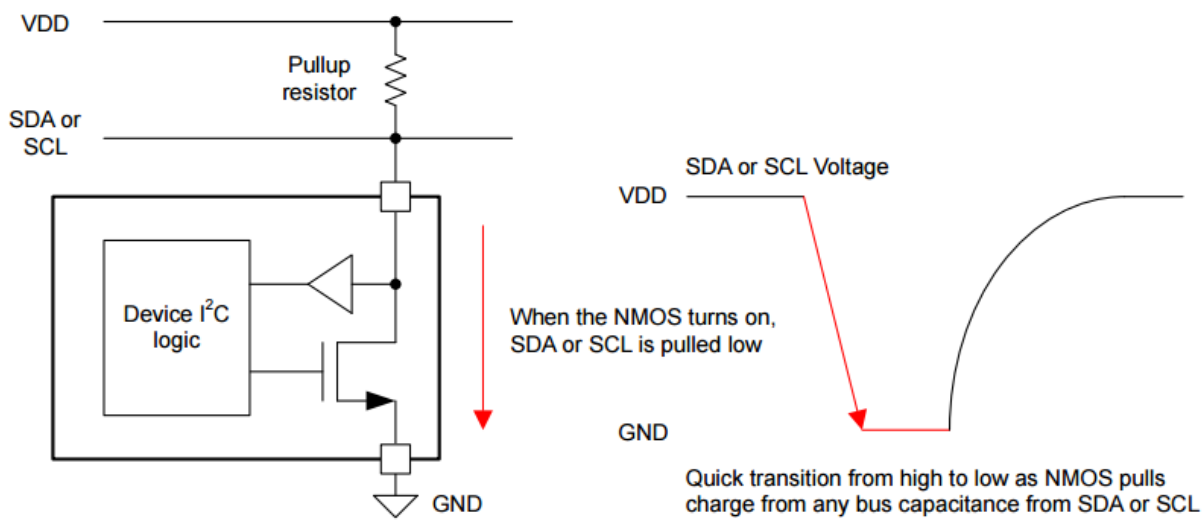



Figure 2: Open-drain I$^2$C wiring model illustrating pull-up resistors and wired-AND behavior.

### 1.1.2 Start and Stop Conditions

I$^2$C defines special bus conditions that frame each transaction. A START condition occurs when `SDA` transitions from high to low while `SCL` is high. This indicates that the bus is being claimed and a transfer is beginning. A STOP condition occurs when `SDA` transitions from low to high while `SCL` is high, signaling that the transaction has ended and the bus is released. These conditions allow all devices on the bus to detect transaction boundaries without any extra chip-select lines.

Because START and STOP are detected while `SCL` is high, I$^2$C requires that data transitions normally occur only when `SCL` is low. This rule prevents accidental interpretation of data changes as control events and ensures that receivers can reliably sample stable data when `SCL` is high.

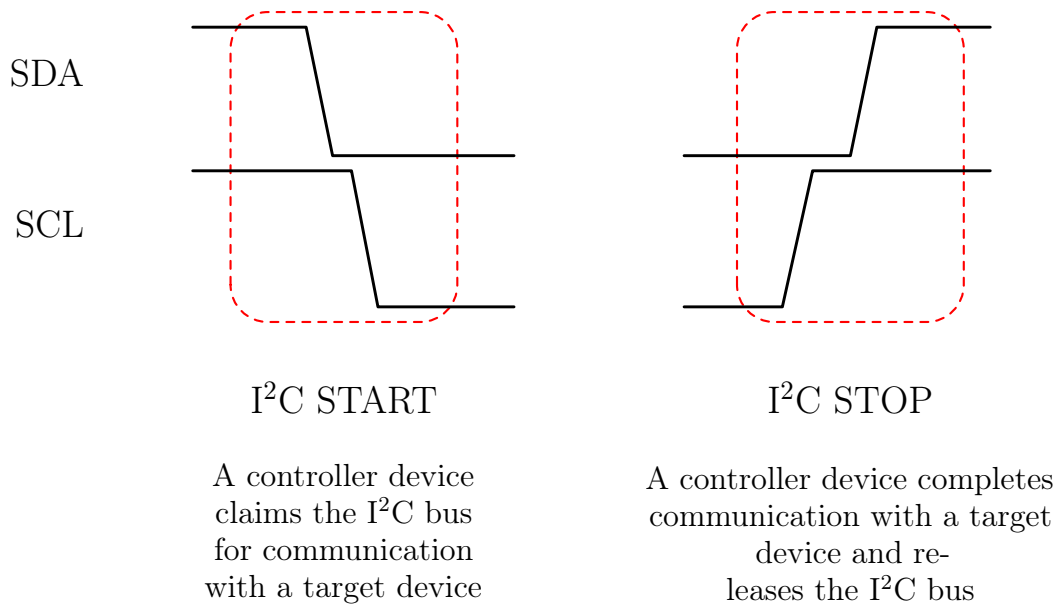



Figure 3: I²C START and STOP conditions defined by `SDA` transitions while `SCL` is high.

### 1.1.3 Byte Transfers and ACK/NACK Handshaking

Data on I²C is transferred in 8-bit bytes, most-significant bit first. After each byte, a ninth clock pulse is reserved for acknowledgment. During this ACK phase, the receiver pulls `SDA` low to indicate ACK, confirming successful reception. If the receiver leaves `SDA` high, it signals NACK, indicating that the byte was not accepted or that no device is responding. ACK/NACK behavior provides immediate error detection and gives the master a mechanism for controlling transaction flow.

This handshake is especially important during reads. The master typically acknowledges each received byte (ACK) while it wants more data, then signals completion by issuing a NACK on the final byte before generating STOP. This pattern prevents ambiguity and cleanly terminates burst reads.

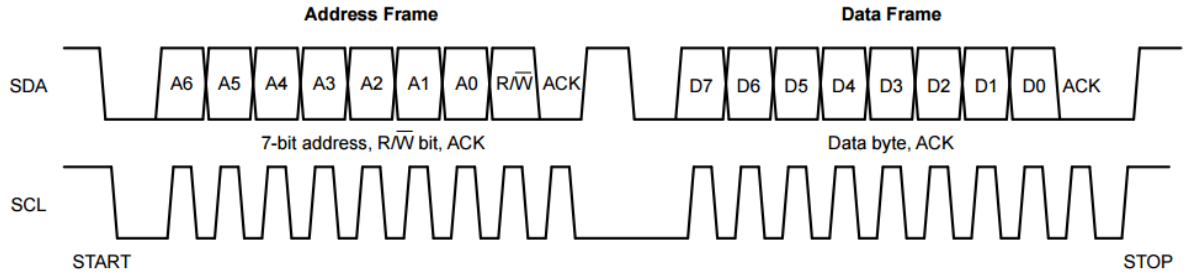



Figure 4: I²C byte transfer showing ACK on the ninth clock cycle.

### 1.1.4 Addressing and Read/Write Direction

After START, the master transmits an address field that selects the target slave device. In standard I²C, the address is 7 bits followed by a single direction bit indicating whether the master intends to

write (0) or read (1). All slaves compare the transmitted address against their own configured address and only the addressed device responds by generating ACK and participating in the remainder of the transfer.

This address-based approach supports multiple devices on the same bus and allows a single master to communicate with each device without separate dedicated wiring. Some systems also use 10-bit addresses for expanded address space, but 7-bit addressing is the most common case for typical board-level peripherals.

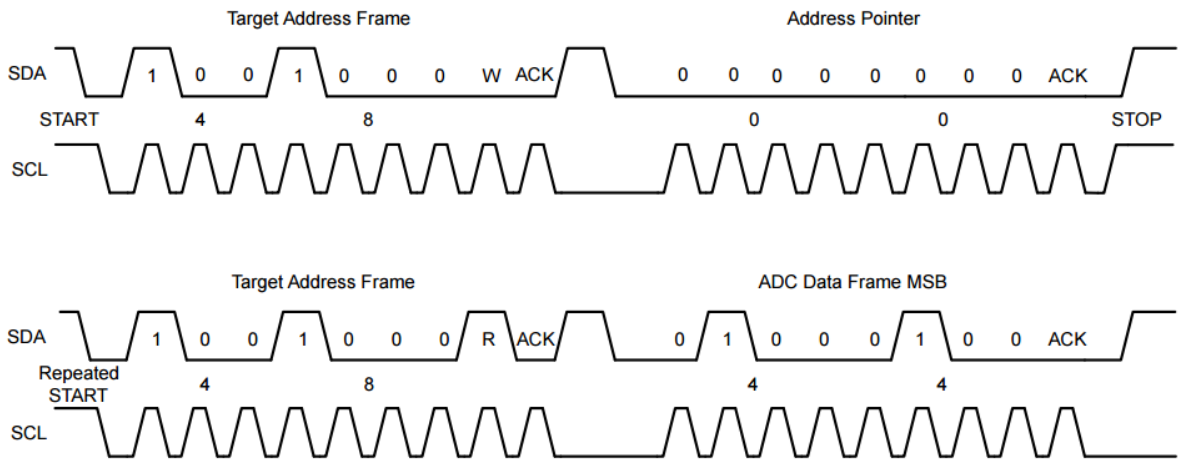



Figure 5: I²C address phase consisting of a 7-bit slave address, R/W direction bit, and ACK.

### 1.1.5 Typical Register-Based Peripheral Transactions and Repeated START

Many I²C peripherals are register-based, meaning configuration and status are accessed by writing or reading internal register addresses. A common pattern for reading a register is a two-step transaction. First, the master performs a write to provide the register pointer (the internal address to read from). Then, without releasing the bus, the master issues a repeated START and performs a read from the same slave address. This repeated START preserves bus ownership and avoids inserting a STOP between the pointer write and the data read, producing a reliable and standards-compliant read sequence.

Write operations are typically simpler: after START and address with write intent, the master transmits a register address followed by one or more data bytes. The slave acknowledges each byte to confirm successful receipt.

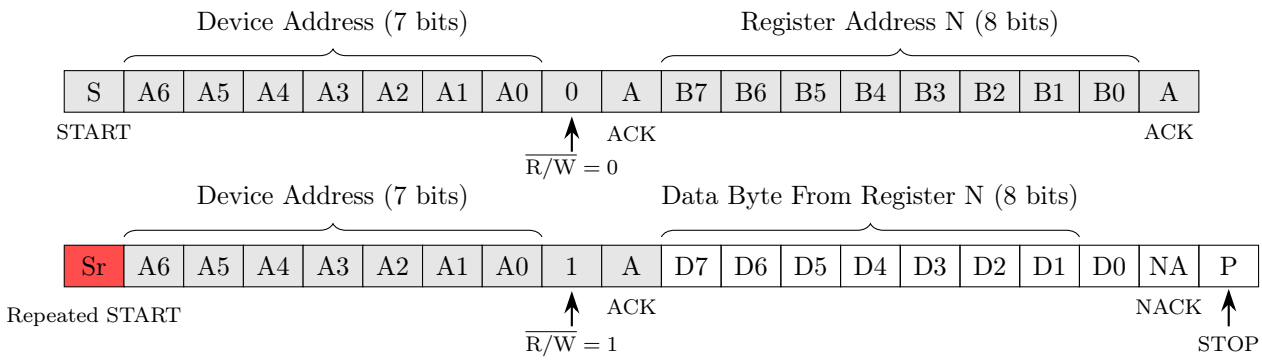



Figure 6: Common I$^2$C register read sequence using a register pointer write followed by a repeated START and data read.

### 1.1.6 Clock Stretching and Multi-Master Considerations

In I$^2$C, a slave device may delay the master by holding `SCL` low, a behavior known as clock stretching. This allows a slower device to pause the bus while it prepares data or completes an internal operation. While not always used, clock stretching is part of the protocol and must be considered when designing a robust controller.

I$^2$C also supports multi-master operation, where more than one master can initiate transactions on the same bus. To prevent collisions, I$^2$C includes arbitration based on wired-AND behavior: if a master attempts to drive a high but observes a low, it loses arbitration and backs off. Many laboratory designs assume single-master operation for simplicity, but the underlying protocol mechanisms enable safe scaling to more complex systems.

### 1.1.7 Finite State Machine Control in HDL

In FPGA-based designs, I$^2$C signaling is commonly implemented using a finite state machine (FSM) that enforces protocol order and timing. The controller transitions through deterministic phases such as idle, START generation, address/byte transmit, acknowledge sampling, repeated START (when needed), byte receive, and STOP generation. Since the FPGA system clock is typically much faster than the I$^2$C bus rate, a clock divider or tick generator is used to create precise `SCL` timing and to ensure correct setup and hold margins on `SDA` relative to `SCL`. This state-based approach makes the design easier to verify with waveforms, supports basic error handling such as NACK detection and timeouts, and provides a reusable communication block for a variety of I$^2$C peripherals.

# 2 Laboratory Structure

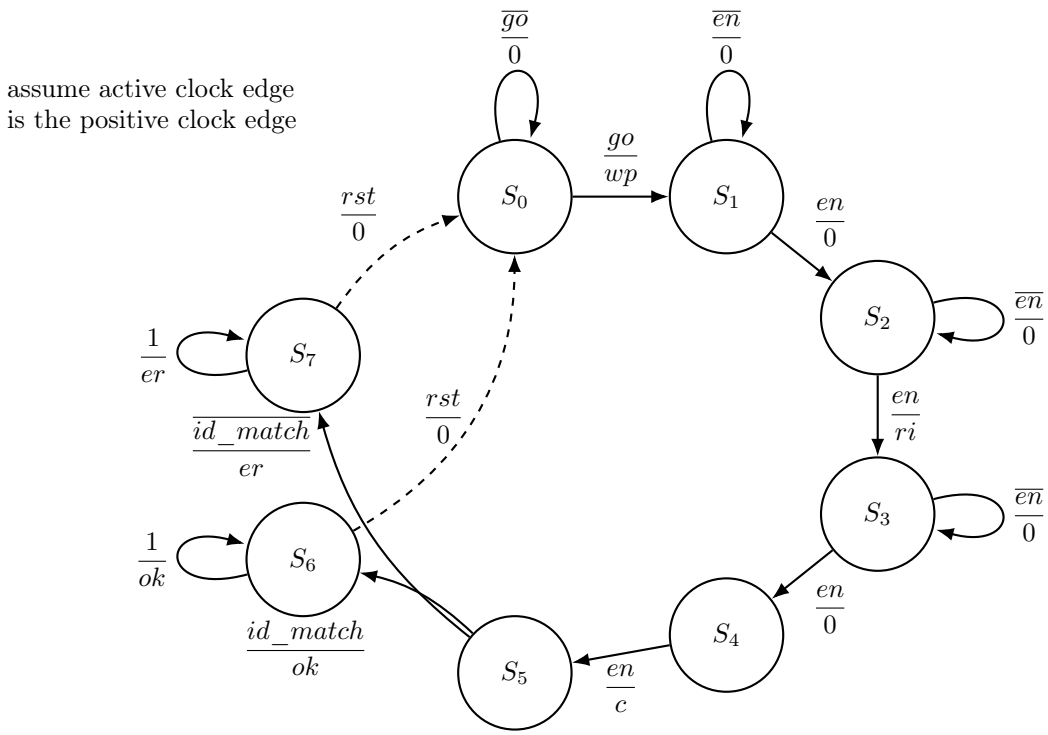
This laboratory is divided into three phases so that students can build the complete design in manageable steps. The goal is to introduce modern peripheral interfacing while keeping the focus on controller design, data handling, and top-level FPGA integration.

### Phase I: I$^2$C Bring-Up and CAP1188 Register Access

In Phase I, your main coding task is to implement the controller in the `send_control_logic` module. The low-level bus timing is already handled for you by the provided I$^2$C support logic, so you should not be writing raw SDA or SCL waveforms in this phase. Instead, your job is to create the higher-level finite state machine that sequences the transaction. Using the extended state graph in Figure 7, you will define the states, state transitions, and output controls that tell the I$^2$C path when to begin a write-pointer operation, when to begin a read operation, and when to compare the returned register value. In other words, this phase is centered on the behavioral logic inside `send_control_logic`, not on the serial bit timing itself.

More specifically, your `send_control_logic` implementation should wait in the idle state until the `go` input begins the transaction. From there, it should assert the control output that starts the write-pointer transaction, wait until the transaction-complete signal indicates that the current bus operation has finished, then assert the control output that starts the read transaction. After the read completes, your controller should enter the comparison state and decide whether the returned data matches the expected CAP1188 register value. Based on that comparison, the design should move into a success state or an error state. In code, this means you will define the state register, next-state logic, and output logic for signals such as `wp`, `ri`, `c`, `ok`, and `er`. The purpose of this phase is to make sure you understand how a controller sequences an I$^2$C register-access operation at the module level.

You should also connect the result of the transaction to a simple visible output so that the controller can be debugged on hardware. For this phase, the easiest method is to display a success or failure condition on the seven-segment outputs using the provided `hex7seg` module. This allows you to confirm that the FPGA is successfully stepping through the register-access sequence before you add any more complex functionality. By the end of Phase I, your code should include a working `send_control_logic` module, the necessary top-level wiring to the provided I$^2$C support path, and a simple visible indication of whether the register access worked correctly.




**Arc Labeling**

$$\frac{\text{Input}}{\text{Output List}}$$

Input = Boolean condition for state transition
Output List = outputs that are Logic 1 during transition
If output is not listed, it is Logic 0
0 indicates all outputs are Logic 0

**Inputs:**
*go* = begin Part I sequence
*en* = I²C operation complete / proceed
*id_match* = read ID equals expected value
*rst* = reset

**Outputs:**
*wp* = start write-pointer transaction
*ri* = start read-ID transaction
*c* = compare received ID
*ok* = device ID valid
*er* = device ID invalid

Figure 7: Extended state graph for Phase I of the laboratory. Students implement this controller behaviorally in Verilog HDL to verify I²C communication through CAP1188 register access.

## Phase II: Read Raw CAP1188 Data and Process It

In Phase II, you will keep the same overall control structure from Phase I, but you will now use it to read meaningful CAP1188 register data instead of only checking whether a known register access works. Figure 8 shows the path that this data takes through the design. The process begins in `send_control_logic`, where your controller starts the read sequence. From there, the controller commands the provided I$^2$C master to perform the required address and register-access transaction with the CAP1188. Once that transaction is complete, the returned register contents become the raw CAP1188 data shown in the dashed block of the figure. This means that the main task in this phase is no longer just sequencing the bus, but also handling what comes back from the peripheral after the read is finished.

As shown in Figure 8, the raw register value should not be treated as the final output of the design. Instead, that value is passed into the decode and processing logic block, where you will write Verilog code to interpret the returned bits. In practice, this usually means masking unused bits, selecting only the fields that matter, or reorganizing the register contents into a simpler internal value that is easier to observe during debugging. After that processing step, the result is sent to `hex7seg`, which drives the seven-segment display. The feedback arrow in the figure also shows that the controller still depends on the captured result to decide whether to continue, compare, or update its operation. By the end of this phase, your code should implement the full path shown in the diagram: `send_control_logic` starts the read, the I$^2$C master communicates with the CAP1188, the raw register data is captured, your decode logic converts that data into a useful form, and `hex7seg` displays the processed value for verification.

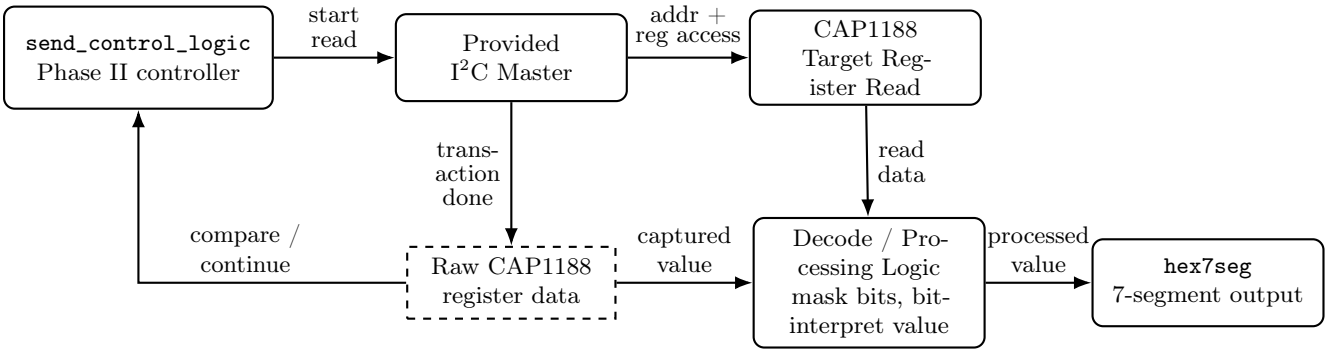



Figure 8: Phase II data flow for reading raw CAP1188 register data, processing it in Verilog, and displaying the result on the seven-segment display.

## Phase III: Display the CAP1188 Result on the LCD

In Phase III, you will connect the processed CAP1188 result to the LCD display path so that the design becomes a complete multi-module system. At this point, the $I^2C$ controller and register-processing logic from the earlier phases should already be working. Your task now is to pass that processed value into the LCD side of the project so that it can be rendered on screen. The LCD hardware path is already supported by the provided modules `tft_ili9341`, `tft_ili9341_spi`, and `tft_ili9341_spi_wrapper`, so your work in this phase is focused on top-level integration and on whatever renderer or text-generation logic is being used in your lab version.

From a coding perspective, this means the value produced by your CAP1188 read path must now be made available to the LCD rendering path. Depending on how your top-level code is structured, this may mean wiring the processed CAP1188 value into a renderer module, selecting that renderer in the display pipeline, or updating the display-control logic so that the LCD shows the current register result or decoded touch status. The important point is that you are no longer working on the communication path in isolation. Instead, you are connecting the communication logic, the data-processing logic, and the LCD output logic into one top-level design. This is the stage where good module boundaries and clean signal naming become especially important, because mistakes often occur when data is moved from one subsystem into another. By the end of Phase III, your system should read data from the CAP1188 through the $I^2C$ control path and display the interpreted result through the LCD interface.

## Optional Post-Lab Questions

1. Why does $I^2C$ use open-drain signaling with pull-up resistors instead of push-pull outputs?
2. What role does ACK/NACK play in making register transactions reliable?
3. Why is a repeated-START commonly used for register reads in $I^2C$ sensors?